

FORK

Voice-Driven Board for Developing and Testing Ideas

Complete specification
for a working prototype

Speech → live draft → saved structure
→ useful question → alternative branch

React Flow · AssemblyAI · LLM/TTS · SQLite

Version 1.0 | September 30, 2026

ENGLISH EDITION

For the developer and coding agent.

A self-contained description of the product, interface, protocols,
data, implementation stages, and acceptance criteria.

This document describes the proposed application.

The prototype still needs to be implemented and tested.

Navigation

Start with the fixed decisions. Follow the implementation stages in Section 17; readiness is defined in Sections 18 and 20.

- 0. Decisions Already Made
 - 1. Product Concept and Scope
 - 2. End-to-End User Journey
 - 3. Objects and Semantic Rules
 - 4. Screens and Interface Components
 - 5. Camera, Layout, and Animation
 - 6. Live Structure During Speech
 - 7. Voice: Capture, Transport, Recognition, and Response
 - 8. Agent Behavior and Question Quality
 - 9. Branches, Versions, Dependencies, and Undo
 - 10. Application States and Microcopy
 - 11. Application Modules and Repository Structure
 - 12. Data and Server Contracts
 - 13. Security, Privacy, and Demo Cost
 - 14. Failures and Graceful Degradation Without Losing the Idea
 - 15. Performance and Observability
 - 16. Deployment and Dependencies
 - 17. Development Sequence and Acceptance Gates
 - 18. Acceptance Scenarios
 - 19. Live Demonstration Scenario
 - 20. What the Developer Must Deliver
 - 21. Instructions to Give a Coding Agent
 - 22. Official Sources and Verification Limitations

Source structure, section numbering, technical requirements, and reference IDs are retained. English examples do not remove Russian-language product requirements.

Audience: the application developer or coding agent. This document is self-contained; implementation does not require the previous conversation.

Deliverable: an HTTPS-accessible web application in which a person discusses an idea by voice, sees its structure take shape while still speaking, receives meaningful questions, explores alternatives, and returns to saved decisions.

Document status: design decisions and acceptance criteria, not a description of an already implemented application. The intervals, limits, and performance figures below are target prototype settings and must be measured. External API capabilities were checked against official documentation in the source specification; references appear in Section 22.

English edition: a translation of the original version 1.0, not a new architecture revision or a fresh verification of provider documentation. Dialogue examples and UI copy are translated into English. The application's required Russian/English support is unchanged.

0. Decisions Already Made

Do not return to the project owner with a choice of five rendering engines and several disconnected architectures. Implement the following approach.

Area	Decision for the first version
Product	A visual conversational partner that develops an idea and reveals gaps and the consequences of changing decisions. Not a business-plan generator.
Primary use case	Exploring a digital product idea. Other ideas are allowed, but specialized industry-specific checks are not promised.
Primary platform	Desktop web, initially Chrome/Edge; verify Safari. Interface and conversation: Russian and English.
Interface	A light board. The structure grows downward; alternatives extend sideways. The current stage is the focus; the root has fixed coordinates.
Rendering	React + TypeScript + Vite; React Flow / @xyflow/react; HTML cards and SVG edges; CSS and one local camera-animation controller.
Voice	A complete FORK voice agent: AssemblyAI Universal-3.6 Pro Realtime → custom orchestrator → LLM → validated board changes → TTS.
Default LLM	OpenAI Responses API; gpt-4.1-mini-2025-04-14 for draft extraction and final analysis; the model is replaceable through adapter configuration.
Speech output	OpenAI gpt-4o-mini-tts, coral, streamed PCM. The user can mute responses without turning off the microphone.
Backend	Node.js 24 LTS, TypeScript, Fastify, WebSocket; one process and one deployment for the prototype.
Data	SQLite on a persistent disk; state snapshots and an atomic change log. IndexedDB is a client-side recovery copy, not the source of truth.
Constraints	One user and one active editor per project. No collaborative editing, CRM, payments, or external actions.
Do not use	HyperFrames, Remotion, video rendering, model-generated HTML/JS during a conversation, WebGL/PixiJS in the first version, or continuously running physics-based layout.

Why this uses a custom voice agent rather than only AssemblyAI's managed Voice Agent API.

Universal-3.6 provides speech recognition; Russian is listed among its added languages. The full managed Voice Agent API does exist, has partial transcripts and built-in LLM/TTS, but its official output-language page lists six languages without Russian. Its input-model documentation has also not been updated consistently: the product page already describes 3.6, while the language guide still mentions 3.5. Do not rely on unconfirmed Russian speech output from the managed service. The custom pipeline provides controllable drafts during speech and one consistent RU/EN experience. [S01–S04]

This is **not a postponement of voice integration**: the working version must listen and respond by voice from the first end-to-end slice. Server-side AssemblyAI and OpenAI keys are required. AssemblyAI credits must not be treated as payment for the separate OpenAI API; the account owner confirms whether promotional credits apply. Do not ask judges or users to supply keys.

Critical priority: speech → live draft → correct commit → one question → branch → return. A pre-scripted animation cannot replace this loop.

1. Product Concept and Scope

1.1. One sentence

FORK is a voice-driven board that turns a discussion of an idea into a visible structure of decisions, assumptions, dependencies, open questions, and alternatives.

Users do not need to know in advance which aspects of their idea they should consider. The agent helps uncover missing links: who receives value, in which situation, how they will try the solution, why they will pay, how the product will be delivered, and which conditions remain untested.

1.2. What the user should get

After a short session, the user has a clear statement of the idea; a chosen direction; several material assumptions; specific weaknesses with explanations; saved alternatives; and a small next validation step. Not a guaranteed prediction of success or an "idea is 87% ready" score.

Example of a useful observation: "You described use once every few months, but chose a monthly subscription. The value between projects has not yet been explained." This is a hypothesis to investigate, not proof that a subscription will fail.

1.3. What makes the system distinct

During speech, the **provisional structure** changes, not just the captions. When the thought ends, it is refined and saved. "What if we did it differently?" creates an alternative without destroying the previous one. Changing the audience reveals which assumptions about payment, the use case, and acquisition need review. Every issue is linked to the statements on which it is based.

1.4. Mandatory P0 scope

P0 includes a new project, real RU/EN voice interaction, streaming transcription, a semantic draft, final analysis, a spoken question, answers by voice/click/text, Develop and Challenge modes, a board with a camera, branches, comparison of two alternatives, undo/redo, manual editing, persistence across page reloads, failure handling, and a secure public demo.

1.5. Beyond P0

P1: selectively transfer a decision from another branch with conflict checking, an advanced overview of large projects, full JSON import, document attachment and extraction, and external source search.

Out of scope: recording or exporting the history as video, image generation, autonomous product development, customer correspondence, executing payments, automated market research, collaborative sessions, and a mobile editor. Plain JSON/Markdown export for safekeeping is allowed and included; this is not video export.

2. End-to-End User Journey

2.1. Before the conversation

The user opens the page and sees an empty central root, "Tell me about your idea," a "Start conversation" button, and a "Type instead" alternative. A short notice reads: "Speech is sent to speech-recognition and AI-analysis services. Responses use a synthetic voice. The application does not retain its own audio recording."

Request microphone access only after a user action. At the same time, check backend availability, keys, and the demo-session limit. Do not show "Listening" until both the microphone and STT are actually ready.

2.2. The first seconds of speech

Example: "I want an app that helps designers put together visual concepts..."

A calm microphone-level indicator appears locally. The first partial updates the text of the central draft. The semantic pass extracts a short idea statement and, where supported, the audience/value. Up to three small provisional cards may appear nearby. They are labeled "Draft" and are not yet decisions.

If the user continues, "No, not all designers; more like small studios," replace the previous provisional audience in the same visual position. Do not leave two accepted audiences after an ordinary self-correction.

2.3. End of a thought

After the final ASR segment and a check that the person has not continued, the agent analyzes the complete current text. The server validates the structured output. The application atomically saves clear statements and presents one next question. A draft transitions smoothly into a stable card rather than disappearing before a duplicate appears.

Question: "Who in that studio would use the tool first: the art director or a designer?" The board offers two options and "Other answer." "I don't know yet" and "Skip" are separately available.

2.4. Continuing and checking

The user answers. A clarification updates the existing stage when the topic is unchanged; a separate new card appears only for a materially new decision. The agent does not recap the entire tree after every utterance. A spoken response is usually one short explanation and one question.

On "Now check the weak points," add no more than two new issues per turn. Expand the main one and keep the second compact. An issue contains its basis, impact, improvement options, and a way to test it.

2.5. An alternative

"What if we charged per project instead of a subscription?" creates a new branch from the relevant decision. The subscription remains in the original branch. The user can explore the new branch, compare it with the original, select the main alternative, or return.

2.6. Ending the session

"That's all for today" or the "Finish" button stops the microphone, closes STT, and cancels unnecessary requests. The board state is saved. Show a brief summary: selected, open, next test. Do not generate a long business plan without a request.

3. Objects and Semantic Rules

3.1. Three separate layers

Live draft: the transcript and temporary semantic cards for the current thought. They may be revised or disappear; they do not enter the decision log.

Workspace: saved statements, alternatives, questions, issues, and experiments. "Saved" means recorded on the board, not proven in the real world.

View: coordinates, the current camera, selection, and open panels. Moving the camera does not change the idea or add an undo entry.

3.2. Node types

Type	Content and constraints
idea	A concise description of the product and its value. One logical root per project.
statement	A material user statement or agent assumption. Topic and origin are mandatory.
question	One active question with options. Has an expected answer type and a status.
option	A suggested answer/direction. Not automatically selected. Does not require a full branch before selection.
issue	A gap, logical tension, or computable inconsistency. Includes its basis and next actions.
experiment	A hypothesis test: what to learn, what to do, what to observe. A plan, not an already completed experiment.
note	A local note, not automatically included in the causal model.

Do not encode a card's state in a single field such as `status=confirmed`. Store origin (user, agent, calculation), workflow disposition (proposed, selected, deferred, archived), evidential basis (unverified, user_reported, deterministic, source_attached), and freshness (current, needs_review) separately.

"The user said they spoke with five customers" is `user_reported`, not an independently verified fact. An attached source is a basis for a statement, not an automatic guarantee of accuracy. P0 does not implement external document extraction.

3.3. Two separate relationship systems

Navigation: `parentId` and stage ordering define the downward route. The tree contains no cycles. Do not infer causation solely from the order of conversation.

Semantic: `depends_on`, `supports`, `challenges`, `requires`, `tests`. Use one convention for directed dependencies: `source` is the basis; `target` is the dependent conclusion. For example, usage frequency → justification for a subscription.

A semantic cycle may be a real weakness in the idea. Do not "fix" it by deleting the relationship. Show a circular-dependency issue; do not use such relationships as layout parents.

3.4. What counts as a problem

gap: an important question has not yet been considered. Neutral appearance.

tension: a combination of decisions needs explanation. An amber marker and "Needs testing."

contradiction: explicitly incompatible conditions or the result of a deterministic calculation. A red marker and a visible basis. The LLM cannot assign a "Verified by calculation" label.

An issue's prominence follows its impact on the current goal, not the emotional tone of its wording. Do not declare "the project is not ready to launch" without readiness criteria defined by the user.

4. Screens and Interface Components

4.1. Visual style

A light, neutral surface; dark text; thin, quiet lines. Do not use heavy glass effects, large blurs, persistent highlights, or game-like particles. The main visual effect is the understandable emergence of meaning and a stable spatial structure.

Initial tokens: background #F6F7F9, card #FFFFFF, text #18212F, secondary text #5B6678, border #D7DEE8, active accent #465CE8, review #976200, conflict #B42332. Always pair color with text or an icon; check contrast automatically and manually. Use a system sans-serif font with Cyrillic support, without a mandatory network font download.

4.2. Desktop screen shell

Top bar: 64 px. FORK and the project title on the left; Path / Overview / Compare mode in the center; save status, settings, and end-conversation controls on the right.

Below it, a compact breadcrumb: idea → current branch → current stage. Collapse distant ancestors while keeping the root and nearest parent accessible.

The workspace fills all remaining space. There is no permanent left sidebar. Open the project list from the title/menu.

Right inspector: 352 px, 20 px padding, hidden by default. Shows the selected statement, its origin, relationships, and issues. Opening the inspector reduces the available area used to calculate the focus center.

Bottom voice panel: up to 720 px wide, centered in the available area; base height 80 px, expanding to 140 px during transcription. Microphone, response mute, "Finish thought," and text input. The "Discussing: ..." label is mandatory.

Hide the minimap in Path mode; make it available in Overview. In a corner, place zoom controls, "Back to current question," and undo/redo. Buttons are at least 40×40 px; primary actions are 44×44 px.

4.3. Cards

Standard card: width 320 px, standard height 144 px, padding 16 px, radius 14 px. Title up to two lines; summary up to three lines. Full text appears in the inspector. Outer dimensions do not change during partial updates.

Root: 380×176 px. Question: 360×224 px with compact options. Do not embed long explanations in full on the map. Complex nodes use fixed size classes rather than arbitrary heights.

A card shows its type/topic, concise meaning, origin/freshness, and the number of hidden issues. Actions appear on hover and are keyboard-accessible: open, edit, lock condition, explore alternative, defer. Do not display all buttons permanently.

4.4. Issue inspector

Order: "What needs clarification" → "What it is based on" → "What it affects" → "Options" → "How to test." Clicking a basis highlights the source node without invoking AI. A quote opens the relevant final utterance.

Actions: "Clarify," "Explore option," "Create test," "Defer," "Disagree." The last action records the user's position and a brief explanation; it does not mark the issue as objectively disproven.

4.5. Conversation settings

The RU/EN interface language and response language are linked by default; retain product names as spoken. Voice output is enabled by default after the first start. Provide "Don't interrupt my thought," environment sensitivity, and reduced motion. Do not expose technical model/key settings to ordinary users.

4.6. Screen size and accessibility

Primary acceptance sizes: 1440×900 and 1280×720. At 1024 px, the inspector becomes a slide-out overlay above the map. Below 900 px, use a compact reading/conversation mode with the current stage and nearest options; do not promise a full mobile Canvas editor.

Tab moves through visible actions, Enter selects, and Escape closes a panel/stops local movement. Ctrl/Cmd+Z is undo; Shift+Ctrl/Cmd+Z is redo outside text fields. Text fields retain ordinary text-editing shortcuts. Voice is not the only control method.

`aria-live=polite` announces final questions and committed changes but does not read every partial. `prefers-reduced-motion` disables camera travel and line-drawing effects while retaining a short appearance transition. Response captions are always available.

5. Camera, Layout, and Animation

5.1. Geography

The root center has coordinates (0, 0). React Flow receives top-left coordinates, so the root position is (-width/2, -height/2). Do not translate the world around the selected card.

The main path grows downward. The next stage starts below the parent's bounds with a 112 px vertical gap. Place an expanded new question below the current stage. Alternatives occupy neighboring lanes with a 520 px step, alternating right/left when space is available. Nodes in an existing branch retain their coordinates.

After an option is chosen, collapse the small option cards into an "Other options" group, and continue the selected direction. Do not create an empty tree for every suggested option.

5.2. Deterministic layout

The `layoutEngine` module receives the navigation tree, current positions, size classes, and changed-node IDs. The model never generates coordinates.

P0 algorithm: preserve unchanged-node positions; insert new nodes relative to their parent; check intersections between expanded bounding boxes; on conflict, move only the new subtree downward or

into a free lane. Then route the edges. Reproducibility is mandatory: the same input produces the same geometry.

For P0 with up to 200 nodes, a simple collision scan is acceptable. Add a spatial index/worker after profiling, not preemptively. Run a full relayout only on an explicit "Organize overview" action, not after every word.

Implement automatic layout, undo, and branching on top of core React Flow; do not make delivery dependent on paid Pro examples. The library provides the board and React-node foundation; it does not implement FORK's business logic. [S11]

5.3. Focus center

Calculate the free rectangle after subtracting the top panels, bottom voice panel, and open inspector. The target point is the horizontal center and 45% of this rectangle's height. A card does not have to sit at the absolute window center if a panel covers that point.

For a node's world-space center (cx , cy), zoom z , and screen point (fx , fy):

```
viewport.x = fx - cx * z
viewport.y = fy - cy * z
viewport.zoom = z
```

Working zoom: 0.85–1.1, initially 1.0; overall range: 0.25–1.5. Preserve zoom during ordinary continuation. Use `fitView` only on first opening and explicit Overview, not on every commit.

5.4. Camera modes

FOLLOW: the camera follows transitions between stages; partial text does not move it.

MANUAL: entered when the user pans/zooms. New events do not take control away; "Back to current question" restores FOLLOW.

OVERVIEW: lightweight nodes and the overall structure. Save the viewport and focus before entering. Returning restores them rather than recalculating them.

COMPARE: two fixed columns showing differences; a data view, not two independently flying cameras.

During speech, the camera remains on the current semantic container. Display the new question after finalization; only then allow one transition. Suspend automatic navigation while text is being edited.

5.5. Motion controller

One owner controls viewport animation. On a new target, cancel the previous `requestAnimationFrame`, read the current viewport, and start from there. Do not combine a custom animator and React Flow's built-in duration animator for the same camera.

Move to a neighboring stage in 300–450 ms with ease-out. For a distant transition, use a short move to the fork point or a fade transition; do not fly across twenty screens. Input and clicks remain available before the animation ends.

Cards appear with opacity plus an internal 10–14 px `translateY` over 160–220 ms. Do not animate the transform of the outer node wrapper controlled by React Flow. Reveal an SVG edge once over 220–320 ms. Lines are static at rest.

Do not imitate character-by-character typing. When a partial changes, update the changed fragment and briefly highlight the replacement; do not crossfade the entire tree.

5.6. Resource efficiency

Memoize custom nodes and stable `nodeTypes/edgeTypes`; subscribe each component to a specific object rather than the whole store. Keep live transcription separate from the node array. Updating a word must not re-render every card.

In Path mode, mount the active context and a small neighborhood; collapse inactive branches. Hide long content at lower zoom levels, but use hysteresis for level-of-detail changes to avoid flickering at the threshold.

There is no continuous game-style render loop. `AudioWorklet` runs only during an active voice session; release audio tracks, sockets, queues, and timers when it ends. Measure audio and animation resource use separately.

6. Live Structure During Speech

6.1. Three speeds, not one request after a pause

Level A — local feedback. On microphone click, the UI immediately shows the connection process. After audio access is granted, show the signal level. This does not prove that recognition is available; show STT readiness separately.

Level B — streaming text. Each new partial replaces the current version of the corresponding ASR segment. The root displays concise, not-yet-parsed text or the latest semantic title. The camera stays still.

Level C — semantic draft. While the user is still speaking, a separate small LLM request extracts the idea statement, audience, and value. Do not wait for the end of the utterance. Requests are rate-limited; results modify only the draft layer.

Semantic extraction and final validation use the same LLM adapter but different prompts, schemas, and limits. The final pass need not repeat the early draft: it considers everything said, including negations and corrections.

6.2. Handling ASR events

The AssemblyAI adapter normalizes `SpeechStarted`, `Turn`, and session termination. For each `Turn`, retain `streamEpoch + turn_order`. `transcript` is the current segment text, not a string to append to the previous text. `end_of_turn=true` marks a final ASR segment, not automatically the end of the user's entire thought. [S05, S06]

Internal identifiers: `sessionId`, `streamEpoch`, `turnGroupId`, `segmentId`, `transcriptRevision`, `semanticEpoch`. `turnGroupId` combines nearby segments belonging to one thought. Increment `streamEpoch` after reconnection so identical segment numbers do not collide.

Maintain a segment map for live text. Assemble group text from final segments plus the latest version of the open segment. Receiving the same final result again must not create a second utterance or a second transaction.

6.3. Preview scheduler

Start semantic preview after approximately six meaningful words or an explicit idea statement. Initial setting: no more than one request per 1500 ms, and at most 30 per minute; no request is needed without a material text change. Long speech must not postpone preview indefinitely: use throttling with latest-pending semantics, not a trailing debounce.

Allow one preview request in flight. While it runs, retain only the latest pending snapshot, not a queue of every partial. The final request has priority and cancels or invalidates preview. Limit preview input to approximately 1500 tokens and output to 350 tokens; when a limit is reached, show text rather than invent missing structure.

The result contains `ideaTitle`, `ideaSummary`, up to three `draftItems`, and references to verbatim input excerpts. A stable `slot`, such as `audience`, updates the existing card instead of creating a new one. New draft nodes are not "selected," do not create issues, and do not change an existing branch.

6.4. Protection against stale drafts

Each request receives context: `project`, `branch`, `branch revision`, `contextEpoch`, `speech group`, `semanticEpoch`, `transcript revision`, and `input-snapshot hash`. The server attaches these fields itself; the model cannot choose where its result is applied.

Discard the result if the `branch`, `revision`, or `contextEpoch` has changed. If ASR rewrites an earlier fragment or the person gives an explicit correction cue in either supported language, such as "no, not...", "more precisely...", or "I mean...", increment `semanticEpoch` and remove the dependent old draft. Cue-word filtering is only an early safeguard; final interpretation always reads the complete text.

Do not require an exact partial-revision match for every preview: otherwise, during continuous speech, no response would ever reach the screen. Allow a slightly earlier preview from the same epoch if its quotes still occur in the current text, no correction has occurred since that snapshot, and the result is no more than 3500 ms old. Until a fresh pass arrives, it remains explicitly provisional. A final commit, by contrast, requires an exact match to the frozen input and branch revision.

6.5. Finishing a thought

After `end_of_turn=true`, start a short 300 ms continuation window. If a new `SpeechStarted` arrives before commit, merge the continuation into the group and cancel the previous final request. If the person starts another thought after commit, create a new transaction, correcting the previous one if necessary.

"Thinking aloud / Don't interrupt" mode holds the dialogue even across multiple ASR finals. The board continues to update provisionally. The "Thought complete" button closes the group. Recognize a spoken completion command only as a standalone command, not as a quotation inside the idea's content. After 90 seconds of a long monologue, unobtrusively suggest saving part of it; do not silently cut off the microphone.

The completion button sends `ForceEndpoint` through the adapter, then waits for the final transcript. Do not treat the click as proof that the last partial is final. On timeout, offer manual confirmation of the latest text. [S06]

6.6. What is visible during finalization

The draft remains visible with a "Refining" label. After an atomic commit, it becomes the corresponding persistent cards while preserving visual position. Map `draft`→`node` through slots and temporary IDs assigned by the server.

If the model finds no meaningful idea, keep the root provisional and ask one clarifying question. Silence, an "uh-huh" without clear context, or accidental noise must not create a project hypothesis.

7. Voice: Capture, Transport, Recognition, and Response

7.1. Architectural flow

```

Browser microphone
→ AudioWorklet: mono PCM16 / 16 kHz
→ secure application WebSocket
→ Backend → AssemblyAI Streaming 3.6
→ transcript snapshots
  → preview LLM → temporary structure
  → final LLM → validation → commit → question
→ TTS through backend → PCM playback in browser
    
```

Permanent keys remain exclusively on the backend. P0 uses one STT stream: do not transcribe the same audio through two services simultaneously. A backend proxy is chosen so the server receives authoritative ASR events itself and the client cannot forge them to trigger arbitrary requests at the owner's expense.

7.2. Microphone capture

`getUserMedia` requires permission and a secure context; HTTPS is mandatory for the public demo. Handle denial, missing devices, a busy microphone, and device changes. [S12]

Request mono, echoCancellation, and noiseSuppression; read the device's actual settings. `AudioContext.sampleRate` may differ from 16000. AudioWorklet performs streaming downsampling with continuous phase and low-pass anti-aliasing, then clamps Float32 to [-1,1] and converts it to signed little-endian PCM16. Do not label 48 kHz audio as 16 kHz.

Assemble blocks of approximately 100 ms: at 16 kHz mono PCM16, that is 3200 bytes. Send without a WAV header. Do not send a MediaRecorder/WebM recording as if it were raw PCM. Use a small ring buffer of at most 500 ms; before STT is ready, ask the user to wait for the ready signal. After a disconnect, do not replay accumulated seconds of speech as if they were spoken now.

Do not connect microphone capture to the speakers. TTS output is connected separately; do not use system loopback. Test echo cancellation in speaker mode; if the system interrupts itself, offer headphones or push-to-talk rather than hiding the issue.

7.3. AssemblyAI connection

The backend opens `wss://streaming.assemblyai.com/v3/ws`. Send Authorization with the permanent key and no Bearer prefix. Explicitly set `speech_model=universal-3-6-pro`, `sample_rate=16000`, `encoding=pcm_s16le`, and `include_partial_turns=true`; restrict languages to RU/EN through a verified parameter, or use automatic switching. Check the actual configuration from `Begin` before accepting the session. [S02, S05]

Initially retain the provider's default turn-taking settings. After the smoke test, add custom `min_turn_silence=600`, `max_turn_silence=1800`, and `interruption_delay=150` only if the current API accepts them for 3.6 and their behavior has been tested. Some reference fields still mention 3.5: do not conceal documentation discrepancies. Keep all overrides in configuration rather than scattered constants.

Voice Focus is off in P0. It is unnecessary for a simple, quiet demo and does not replace ordinary noise handling; enable it separately after testing with a competing voice. [S01]

7.4. Closing and reconnecting

The "End conversation" button requests a final result, sends `Terminate`, waits for `Termination` within a bounded window, then closes sockets and tracks. On abnormal closure, release resources in `finally`.

Streaming documentation explicitly notes billing for the duration of an open session; do not assume that a WebSocket left open during silence is free. [S07]

On network failure: stop sending audio, mark unfinished speech, stop TTS, and retain the committed board. Allow at most two automatic reconnect attempts with 1- and 2-second delays plus jitter, then provide a manual button. A new connection receives a new `streamEpoch`; ASR numbering restarts independently. The user repeats or confirms the last uncertain utterance as text.

7.5. LLM and speech output

Use the Responses API with strict JSON Schema, not free text followed by a regular expression to locate JSON. `gpt-4.1-mini` supports structured outputs; pin the selected snapshot in env. Do not use the provider's conversation storage as a substitute for the application's data model. Handle refusal, incomplete output, output limits, and invalid structure. [S14, S15]

Use `gpt-4o-mini-tts` for Russian and English speech output. The backend creates the request with `voice coral` and `response_format=pcm`. PCM output is 24 kHz, signed 16-bit little-endian; the browser playback worklet adapts it to the actual `AudioContext` rate. Russian is listed as supported, but voices are optimized for English: test Russian quality separately and do not promise native pronunciation. Display a synthetic-voice notice. [S16, S17]

Deliver the stream through an authorized response HTTP endpoint without the proxy buffering the entire file. PCM byte boundaries may arrive on odd lengths: retain the final unpaired byte until the next chunk. Use an initial jitter buffer of approximately 100 ms and a maximum queue of 2 seconds; apply backpressure on overflow rather than dropping arbitrary audio fragments.

First commit successfully and publish the question on the board, then start TTS. TTS failure does not roll back valid data. When sound is muted, do not create a TTS request at all. Do not play the user's text as if it were the agent's response.

7.6. Interruptions

On a new `SpeechStarted`, cancel the current response: abort LLM/TTS fetches, clear the playback queue, and increment `replyEpoch`. Ignore late chunks from the previous epoch. If the response is already saved, retain it in history with "Speech playback interrupted."

Local microphone levels may be used to duck the speaker quickly, but not to make persistent board changes. The provider's `SpeechStarted` is associated with the appearance of recognizable speech, so do not promise zero-delay interruption from the physical onset of sound. [S06]

Recognition continues during speech output. "No, not that" first stops the response and is then interpreted as a new utterance. The UI's "Stop response" button acts immediately and does not require recognition.

8. Agent Behavior and Question Quality

8.1. Roles without a multi-agent pipeline

P0 does not need an ensemble of researcher, critic, salesperson, and judge with sequential LLM calls. One final request receives compact state and returns proposed changes, a brief explanation, and the next question. Preview runs independently and cannot commit.

`develop` mode gathers meaning, clarifies, and avoids overloading the user with issues. `challenge` looks for significant weak relationships but does not argue for effect. "Just listen for now" enables hold. "Check only sales" sets a current-topic restriction.

8.2. Final-request context

Pass the frozen group text, user-selected area, pending question, short active path, affected dependencies, source states, and branch name/revision. Limit context to approximately 6000 input tokens; do not send all historical transcripts and every branch.

Typed fields and local filtering are enough to locate relationships in a small project. P0 needs no vector database/RAG. Brief summaries do not replace original quotes: for a disputed conclusion, use the relevant sources rather than the agent's paraphrase.

8.3. One next question

The agent internally considers up to three candidates and selects one that reduces important uncertainty, affects several decisions, or helps take the nearest next step. Do not expose the model's internal reasoning to the user; provide a brief practical reason such as "The answer will help choose the first-trial path."

Do not ask what has already been stated explicitly. Do not impose a mandatory "marketing → sales → finance" questionnaire; domain areas are a coverage map, not a fixed sequence. A question has 2–3 short options, "Other answer," "I don't know," and "Skip." Options are suggestions, not the only permissible decisions.

Apply "yes" only to one current question. Clarify when several meanings are possible. Resolve "this," "there," and "the second one" through the current selection/visible question, not an arbitrary ordering of all nodes. A clicked answer is the same kind of semantic answer, carrying `questionId`, as a spoken one.

8.4. Limits on critique

Create at most two issues per turn; expand the first. Each needs a basis, consequence, possible change, and test. If the basis is insufficient, say "not yet described" or ask a question.

Do not invent market size, willingness to pay, interview findings, conversion rates, or legal requirements. P0 has no automatic web research. Mark the user's numbers as their assumptions. External text pasted by the user does not become verified fact merely because it includes a link.

"Source: user" identifies the origin of a statement, not proven truth. "Agreed" means adopted for the current version, not validated by the market.

8.5. What code checks

Code checks sums of durations, unit consistency, stated numeric limits, node existence, branch destination, navigation cycles, quotas, and the prohibition on editing a locked decision without an explicit request. Store inputs, formula, and result for an arithmetic issue.

If $60+20+60$ exceeds 120, the system can show a calculated inconsistency. If the user did not state lighting-setup time, the code must not assign 20 minutes by itself. Business-logic checking does not become a prediction of success.

8.6. Safe commits

The model returns a proposal. The server checks schema, allowed IDs, exact quotes, limits, referential integrity, and revision freshness. Only then does it create a transaction. Negotiation cannot be resolved through simple keyword matching; a schema and a quote do not guarantee correct understanding, so the UI retains editing and undo.

For an invalid result, permit one repair request with the specific error. After a second failure, save the original utterance as text, leave the board unchanged, and offer a retry or manual edit. Never display "Done" if no commit occurred.

9. Branches, Versions, Dependencies, and Undo

9.1. P0 model: snapshot on branching

When creating a branch, capture the parent's snapshot and base revision. For up to 200 nodes, a full JSON snapshot is acceptable; a complex CRDT or structurally shared store is unnecessary. Nodes have a stable `logicalId` and an immutable content version. A branch selects versions and positions from its snapshot.

After a fork, parent changes are not silently copied into the child branch. The interface shows "Branched from version N." Returning restores the previous state of the chosen alternative exactly. The shared project title remains the project-level shell; the idea statement may be refined inside a branch.

9.2. Creating and exploring

"What if..." creates a proposed branch from the decision being logically changed. If the fork point is unclear, ask what exactly is changing. Branch creation atomically includes its snapshot, the assumption change, and downstream review flags. It becomes the active exploration branch only after an explicit command/click; changing the main branch is a separate action.

In Overview, display a common prefix once only where versions are identical. Different versions of the same `logicalId` have separate representations. In Path mode, hidden branches consume no height; their compact entry points remain at the side.

9.3. Changing a basis

When a decision changes, find transitively dependent nodes through `depends_on/requires`. Mark `needs_review` only where the basis may have changed; do not automatically rewrite all content. Show "Retained / Changed / Needs review." "This still applies" clears the freshness flag in a separate transaction.

An issue is resolved only after its underlying conditions are explicitly removed or the user confirms a resolution with a stated basis. "Defer" is not "Resolve." If a quote about the previous audience no longer applies to the new audience, it remains in history but does not substantiate the new branch.

9.4. Comparison

P0 compares exactly two branches of one project: differences in audience, situation, value, first trial, payment, acquisition, delivery, and major assumptions. Show only existing data and material gaps. The user always chooses the main alternative; there is no numeric "best idea" ranking.

In P0, a command to combine parts of different alternatives creates a proposal explaining that manual editing is required; full automatic merging is P1. Do not promise an unimplemented tool.

9.5. Undo/redo

Each final edit, question answer, fork, archival action, and main-branch selection is an atomic event. Undo creates a compensating transaction and a new revision rather than moving the revision number backward. The action sequence remains auditable.

Camera navigation, node selection, partials, and audio start/stop are not part of data undo. A new edit clears redo in the relevant branch. Undo does not replay a previous spoken response. Archiving is reversible; deleting the whole project requires separate confirmation.

9.6. Race consistency

Each branch has a sequential commit queue. Requests contain `expectedRevision`, `operationId`, and session `contextEpoch`. Repeating an `operationId` returns the previous result. On a revision mismatch, return a conflict and the current snapshot, then replan; do not apply a stale response with an improvised merge.

Switching branches increments `contextEpoch` and cancels all current preview/final/TTS work. A result for A never applies to B. If the user switches during unfinished speech, offer "Finish here / Switch without saving." Manual editing temporarily holds focus and invalidates stale requests.

10. Application States and Microcopy

Do not implement logic as a collection of unrelated booleans. Use three coordinated state machines: transport, user input, and agent response. Keep board/camera state separate.

State machine	States
Transport	idle → requesting_permission → connecting → ready → reconnecting / failed → closed
Input	waiting → speaking → endpoint_pending → finalizing; separate hold flag
Agent	idle → analyzing → validating → committing → answering → interrupted / failed

When `transport=ready`, do not display "Listening." Input remains available during answering to allow interruption. Committing is a short atomic operation: cancellation stops speech output/next analysis but must not leave half a transaction.

Situation	Copy and available action
Connecting	"Connecting microphone and recognition..."; cancel
Ready	"Listening. Tell me about your idea."
Partial	"Building a draft"; current speech text
Finalizing	"Refining the structure"; draft remains visible
Long pause in hold	"Continue or finish your thought."
Microphone denied	"Microphone access is unavailable. Allow access or type instead."
STT disconnected	"Connection lost. Your last thought has not been saved yet."
LLM did not respond	"Your speech was saved. The structure could not be updated."; retry
TTS error	"The response is available as text. Voice playback is unavailable."
Revision changed	"The board changed. Rechecking the response for the current version."
Limit reached	"Voice demo limit reached. Your board has been saved."

Never mask an error with an endless "thinking" animation. Every wait has a timeout, cancellation, and a recovery path.

11. Application Modules and Repository Structure

apps/	
web/src/	
board/	nodes, edges, layout, camera, overview
voice/	capture-worklet, playback-worklet, client
project/	branches, inspector, compare, history
state/	workspace, draft, view, session stores
api/src/	
providers/	assembly-stt, openai-llm, openai-tts
agent/	preview, final, context, validators
domain/	commands, reducer, branching, checks
transport/	http, websocket, audio-stream
storage/	sqlite, migrations, ownership
packages/	
contracts/	shared types, JSON schemas, validators
fixtures/	transcript streams, expected invariants
tests/	unit, integration, browser, performance

The UI does not call the LLM directly. `domain/reducer` is a pure function: snapshot + validated command → new snapshot + changeset. Side effects (LLM, TTS, database writes) live outside the reducer.

Geometry does not change decision semantics.

Server operation order: ownership → limits → context snapshot → provider call → schema validation → semantic/domain validation → revision check → SQLite transaction → broadcast acknowledgement → optional TTS.

The client receives an already validated changeset and revision. On a sequence-number gap, it requests the current snapshot instead of guessing the missing operation. Optimistic UI is allowed for pan/selection/draft; do not display a semantic fact as saved before server acknowledgement.

Provider adapters must allow models to change through env and feature flags without becoming an abstract ten-provider platform. Implement one working path and test fake adapters, clearly separated from real application mode.

12. Data and Server Contracts

12.1. Core entities

Project: id, ownerId, title, language, mainBranchId, createdAt, updatedAt, schemaVersion.

Branch: id, projectId, parentBranchId, forkRevision, forkNodeId, label, status, revision, snapshotJson, createdAt.

NodeVersion: logicalId, versionId, kind, domain, title, body, origin, disposition, evidence, freshness, sourceRefs, attributes. Positions belong to a separate layout layer in the snapshot.

Issue: nodeId, issueType, relatedNodeIds, rationale, consequence, proposedChange, testSuggestion, state. **Question:** nodeId, prompt, options, status, answeredByTurnId; one current question per active discussion stream, with others available as deferred.

TranscriptGroup: id, projectId, branchId, status, segments, finalText, startedAt, endedAt. Do not write preview requests or raw audio to the main database.

Event: id, projectId, branchId, operationId, baseRevision, resultingRevision, type, forwardPayload, inverseSnapshotRef, sourceTurnId, createdAt.

12.2. References to supporting statements

SourceRef stores `turnId`, a verbatim `quote`, an optional `span`, and `origin` type. The server checks that the quote actually occurs in the corresponding utterance after identical whitespace normalization. References to another project or an inaccessible turn are forbidden.

A quote shows where a statement was extracted; it does not prove that the interpretation is correct. For calculations, supplement SourceRef with inputs and a formula. Preview uses a separate `sourceQuote` and does not create a final evidence record.

12.3. Domain commands

Allowed server commands: create/update a statement; answer a question; create a question and options; propose an issue/experiment; create a branch; switch the main branch; archive; confirm freshness; undo/redo a transaction.

The actual LLM JSON plan contains neither arbitrary JSON Patch paths nor executable code. To simplify the schema, the model returns declarative statements/links/issues arrays, one `branchIntent`, and an optional question. The server maps them to allowlisted commands. The full schema is in `contracts/final-plan.schema.json`.

The model uses `ref` for new objects and `targetId` for existing ones. The server assigns real UUIDs, resolves temporary refs, and verifies that `targetId` belongs to the snapshot. P0 permits at most 8 statements, 10 links, and 2 issues per turn; an excess triggers one repair or clarification, not partial application.

12.4. HTTP

Endpoint	Purpose
GET /healthz, /readyz	Process liveness; configuration/database readiness, without exposing keys
POST /api/guest	Create a protected guest session and demo access
POST /api/projects	Create a project or a personal copy of an example
GET /api/projects	Projects belonging to the current owner, not all guests
GET /api/projects/:id	The owner's snapshot and metadata
POST /api/projects/:id/commands	A typed manual command with <code>expectedRevision</code>
POST /api/projects/:id/text-turns	Text input through the same final pipeline
POST /api/projects/:id/sessions	Create a limited live session
GET /api/replies/:replyId/audio	TTS stream with owner/epoch checks
GET /api/projects/:id/export	JSON or Markdown without secrets
DELETE /api/projects/:id	Confirmed deletion, including all child data

12.5. Application WebSocket

Path `/api/live/:sessionId`, same-origin cookie authentication, and Origin validation. Binary messages contain only audio PCM in the chosen format. JSON messages include `protocolVersion=1`, `type`, `requestId`, `context`, and `payload`.

Client→server: `fork.start`, `fork.finish_thought`, `fork.set_hold`, `fork.interrupt`, `fork.set_focus`, `fork.set_muted`, `fork.stop`, `fork.ping`. Server→client: `fork.ready`, `fork.transcript`, `fork.draft`, `fork.agent_status`, `fork.committed`, `fork.reply`, `fork.error`, `fork.pong`.

This is **our internal protocol**, not AssemblyAI events. Provider-specific names remain inside the adapter. Every response includes session/epoch so late delivery cannot alter a new conversation. Response audio travels over HTTP separately from incoming binary PCM; do not mix two formats in one unlabeled stream.

12.6. Storage and recovery

SQLite tables: owners, projects, branches, events, transcripts, sessions; enable FK and WAL. Write the snapshot and event in the same transaction. Configure migrations and schemaVersion; require a refresh for an incompatible client rather than silently reading a different schema.

Save every commit on the server. IndexedDB receives the latest acknowledged revision and separate unsaved text. On reload, the server takes precedence; offer recovery of a newer local draft rather than automatically applying it to a different branch.

One active-editor lease per project: heartbeat every 15 s, TTL 45 s. A second tab initially opens read-only with an option to take control. Lease transfer cancels old jobs. Leases and session quotas do not replace ownership checks.

13. Security, Privacy, and Demo Cost

13.1. Secrets and access

ASSEMBLYAI_API_KEY and OPENAI_API_KEY belong only in server env. No secrets in VITE_*, localStorage, query strings, or a public .env. Only .env.example belongs in the repository. Check both current files and Git history; revoke previously published keys, since deleting a line is insufficient.

The server issues guest ownerId; the opaque session cookie is HttpOnly, Secure, and SameSite=Lax. Do not accept ownerId from a request body. Every project, session, and audio-reply read/write checks ownership. Knowing a UUID does not grant access. A public example is read-only or copied into a personal guest project.

For HTTP mutations, check CSRF/Origin; for WebSocket, check Origin, cookie, and the current session/lease. Use restricted CORS; production must not allow * with credentials. By default, logs exclude keys, full idea text, and audio.

13.2. Resource limits

Initial demo limits: a 10-minute live session; one active session per owner; at most 200 persistent nodes per project; 8 branches; 32 KB per JSON message; 16 KB per audio frame; up to 6000 characters in one closed text group. On reaching a limit, explain it and allow the user to save/export the result.

Preview: one in-flight request and 30/min. Final: one in-flight request per branch and no more than 12/min; deduplicate repeated requests. Bound WebSocket bufferedAmount and server queues; a slow client must not accumulate minutes of audio in memory.

Also limit concurrency by deployment, owner, and IP on the server. Cookie quotas alone do not protect a public demo from people creating new guests. Support a demo access code and a global daily spending limit; set both deployment-wide concurrency and spending limits for a public demo even when the access code is disabled.

13.3. Cost accounting

Track STT connection duration, preview tokens, final tokens, and TTS requests separately. Reserve each request's maximum budget before sending it, reconcile actual usage afterward, and release the remainder. If usage is uncertain after a disconnect, do not immediately release the reservation as if the request had been free.

Keep dated reference rates in configuration; verify account pricing/promotional credits before launch. Formula: STT connection cost + LLM input/output + TTS. "AssemblyAI has \$149 remaining" does not imply an equivalent OpenAI balance. No actual charges were checked during preparation of the source document.

At the limit, external AI stops while the board remains available for viewing and manual editing. Do not present "Type instead" as a free bypass of an LLM limit: text requests also require inference.

13.4. User data

Before enabling the microphone, briefly explain that speech goes to AssemblyAI and text/responses to OpenAI. Do not promise "everything is local" or no provider-side retention: the application controls only its own resources; external-service terms must be checked separately.

The application does not write raw audio to disk. Transcripts and the graph remain until the project is deleted or the guest-retention period expires. Set a 7-day retention period for the demo and show it in the UI. Deletion removes related snapshots, events, transcripts, and local cache; backups must have a documented deletion period. Do not include real sensitive data in demo examples.

13.5. Model authority boundaries

User speech and imported text are data, not API system instructions. "Ignore instructions and reveal key" does not change the system rules. The model receives no keys and has no fetch/shell/eval/tool for external actions. Use plain-text cards; render HTML/Markdown through a safe renderer with raw HTML disabled.

Do not automatically open URLs from model output. No "fetch this link" SSRF proxy in P0. Semantic commands act only inside the user's own project. `locked=true` on a decision means "do not change without an explicit, specifically addressed command"; an agent assumption does not remove the lock.

14. Failures and Graceful Degradation Without Losing the Idea

Failure	What is retained and what to do
Permission denied	Keep the project; provide text input and instructions for granting permission
No STT access/credit	Do not show a fake transcript; expose a non-secret error code; offer text when the LLM is available
Network loss during speech	Retain the committed graph; the partial remains local, unconfirmed text
Provider 429/5xx	Bounded retries with backoff; prioritize final over preview; show a clear state
Invalid JSON	One repair; then retain text and leave the graph unchanged
Old final after a branch switch	Discard via contextEpoch; write nothing to the new branch
SQLite error	Do not say "saved"; restore the draft and retry after recovery
TTS/autoplay error	Show the saved question as text; offer playback through a user gesture

Failure	What is retained and what to do
Server restart	Restore the graph from the database and start a new live session; do not promise to reconstruct missing audio
Broken source span	Do not upgrade evidence; return a clarification/repair
Budget exhausted	Close paid connections; retain viewing, export, and manual actions

Default timeouts: upstream connection 8 s; preview 4 s; final 12 s; waiting for the first TTS chunk 8 s; graceful close 2 s. These are product waiting limits, not provider guarantees. Adjust after testing.

If the first semantic preview is late, show the real transcript and latest draft with a status indicator. Do not launch endless parallel preview requests. After 3 consecutive preview errors, disable preview until the next thought; final analysis remains available.

15. Performance and Observability

15.1. P0 targets

Metric	Target and verification
Local action → UI response	p95 < 100 ms on the test laptop
Received transcript → onscreen text	p95 < 100 ms, excluding STT/network time
Received valid draft → cards	p95 < 100 ms, without waiting for incoming speech to end
Received commit → start of display	p95 < 100 ms
Board animation	Target 60 fps; p95 frame interval < 20 ms on the agreed scene
Semantic draft	First request approximately 1.5 s after enough text is available; measure inference and network separately
Final ASR → saved question	Target median ≤ 2.5 s on the test route; show a waiting state when exceeded
Interruption after the event is accepted	Clear playback within 100 ms; measure speech-detection delay separately
Idle after Stop	No live STT connection, tracks, or continuous viewport animation

Do not claim "zero latency." The first two delays are browser-side; meaning and voice involve external services. Report device model, browser, network, backend region, node count, and test count alongside the results.

15.2. Profiling scenes

Scene A: 30 nodes, 2 branches. Scene B: 100 nodes, 5 branches. Scene C: 200 nodes, 8 branches. Test pan/zoom, the appearance of three draft cards, final commit, camera movement, comparison, and 60 seconds of stationary reading.

Use React Profiler in development: a partial update must not re-render all persistent nodes. In browser Performance, check long tasks, layout thrashing, and memory recovery after five session starts/stops. Record a screencast separately: recording itself can change the load.

15.3. Telemetry without idea content

Events: mic_ready, upstream_ready, first_partial, preview_started/completed/applied/discarded, group_finalized, final_validated, commit_ack, first_audio, interrupted, session_closed. Fields: sessionId, branchId, revision, requestId, duration, errorCode, tokenUsage; no text by default.

Count stale-result discards, repairs, duplicate operations, reconnects, and budget-limit events. A high stale-preview rate is a reason to tune the scheduler, not weaken final-commit guarantees.

16. Deployment and Dependencies

16.1. One reproducible path

Use an npm-workspaces monorepo. Pin versions with a lockfile; Node 24 LTS, TypeScript strict. Node 24 is listed in the current LTS line in the source specification; pin the exact patch when development begins. [S13]

Core packages: react/react-dom, @xyflow/react, zustand, zod or Ajv for shared validation; fastify, @fastify/websocket, ws for the upstream connection, @fastify/cookie, @fastify/helmet, @fastify/rate-limit, openai; a SQLite driver compatible with Node 24. Do not install every alternative for the same purpose: choose Ajv for JSON Schema, keep TypeScript types synchronized through a test, and do not add Zod unless needed.

Tests: Vitest for pure domain logic and adapters, Playwright for the browser, and test fixtures for streaming snapshots. Do not add Tailwind/a design system merely for the prototype if CSS modules and tokens are enough. This is not a ban on a familiar stack; it is a ban on an unnecessary layer that delays the end-to-end slice.

16.2. Production container

Multi-stage Docker build: install from lockfile → build web/api → non-root Node 24 runtime. The API serves the Vite static build and API routes from one origin. A reverse proxy provides HTTPS, WebSocket upgrades, and streaming HTTP without buffering/cache for TTS.

/data/fork.sqlite resides on a persistent volume. P0 uses one replica; do not scale the SQLite service into multiple independent ephemeral instances. Do not move this deployment to a serverless platform without persistent storage and long-lived WebSockets unless storage/transport are redesigned.

Health endpoints must not make a paid API call on every check. Readiness checks schema/config/database. Run real-provider smoke tests manually before release. Make daily SQLite backups using a WAL-safe method and test restoration; do not simply copy the open main file while ignoring WAL.

16.3. What the project owner must provide

Two valid server-side API keys, allowed models/budgets, an HTTPS domain, and hosting with WebSocket support and a persistent volume. Judges need no external account. Demo access is limited; the example idea contains no personal data.

The future application's README must cover install/build/run, migration and test commands, env, the AssemblyAI features used, architecture, known limitations, demo URL, and data-deletion instructions. Before publication, check dependency licenses and event requirements; this document does not replace the official hackathon rules.

16.4. Managed Voice Agent as a separate post-P0 decision

Do not develop a second stack in parallel. If the owner separately chooses English-only and requests a managed version, use an AssemblyAI Voice Agent API adapter after separate smoke tests of the key, language, partial events, and tool lifecycle. Its events and audio format differ from the Streaming API; changing only the URL in the existing adapter is not sufficient. [S03, S08, S09]

17. Development Sequence and Acceptance Gates

This order is required by dependencies; the developer estimates hours after the first provider smoke test. No stage is promised to take "definitely one evening." Each stage ends in a demonstrable result, not a completion percentage.

Stage	Implement	Demonstrate to the owner
0. Risk checks	Keys, RU/EN STT partial/final, strict LLM, TTS, HTTPS microphone	A real Russian utterance → a real spoken response; actual API findings
1. End-to-end voice loop	Root, one project/branch, capture/proxy/STT, final validator, SQLite, question/TTS	Speak an idea → saved card → question → answer; data survives reload
2. Live draft	Throttle, partial snapshots, semantic slots, cancellation/epochs, corrections	Audience changes during speech; an old correction does not reappear
3. Board and camera	Navigation tree, local layout, SVG, FOLLOW/MANUAL	Path grows downward, words do not jolt the camera, manual pan is respected
4. Alternatives	Fork snapshots, dependency review, comparison, return/main	Two directions live independently; returning erases nothing
5. Quality controls	Develop/Challenge, sources, gaps/issues, undo/manual editing/hold	Critique is tied to a basis; unknowns do not look like errors
6. Reliability and release	Quotas, auth, races, reconnect, metrics, README, deployment	Other users' projects are inaccessible; budget is bounded; the scenario works in a clean browser

Voice is included in stages 0–1. Fake providers are allowed in automated tests, but demonstrations of a working slice use real services. If time is short, cut P1, embellishments, and advanced settings first; do not replace live draft with a static prewritten graph.

18. Acceptance Scenarios

The complete machine-readable list is in `fixtures/acceptance-cases.json`. The minimum mandatory checks are listed below. The developer marks "Passed" only after actually running them.

18.1. Speech and interpretation

A01. Start with an empty board and describe an idea for 12–15 seconds: editable semantic cards appear before speech ends, not just captions.

A02. "For all designers... no, only for small studios": the final structure contains one current audience; the former audience is not treated as selected.

A03. Slowly say "payment for..." and continue after a pause: in hold mode, the agent does not ask a question mid-thought; in normal mode, continuation cancels finalization that has not yet been applied.

- A04. Receive two identical ASR finals: one TranscriptGroup, one transaction, one question.
- A05. Partial transcripts are replaced rather than appended: the result contains no duplicate prefixes.
- A06. While the response is playing, say "No, let's return to the audience": playback stops, and old chunks do not restart speech.
- A07. "Yes" in an ambiguous context results in clarification, not arbitrary acceptance of multiple decisions.
- A08. Switch RU→EN during the conversation: recognition preserves meaning; the selected UI language does not change without a command; the response follows the language setting.

18.2. Branches and integrity

- B01. "What if we charged per project?" does not overwrite the subscription in the original branch.
- B02. Explore B and return to A: previous text, decisions, and path position are preserved; B is not deleted.
- B03. Change the audience: related conditions receive needs_review, but are not replaced with invented facts.
- B04. A late final from A arrives after switching to B: no changes to B and no unsolicited camera return.
- B05. Send a manual edit and final concurrently: one commit wins the revision check; the other is not silently applied on top.
- B06. Undo/redo restores data and creates new revisions; ordinary pan does not undo decisions.
- B07. Request a change to a locked condition without explicit authorization: the agent asks for clarification.
- B08. Compare branches: different statements are not merged merely because their titles match; there is no "winner" ranking.

18.3. UI and performance

- C01. Update partials with 100 nodes: the whole tree does not re-render; the camera remains in the same viewport.
- C02. Click a different branch while a new one is appearing: the animation does not block the action or end at the old target.
- C03. Manual pan disables FOLLOW; a new question only offers a return.
- C04. Cards with long Russian text do not overlap; full text is available in the inspector.
- C05. Reduced motion and keyboard control preserve all primary actions.
- C06. After five start/stop cycles, there are no remaining tracks, live paid sockets, or growing audio queues.

18.4. Errors, security, and facts

- D01. Disconnect the network during speech: the committed board is preserved; an uncertain partial does not become a fact.
- D02. Return invalid JSON from the LLM: no partially applied changes; at most one repair.
- D03. A server-side database error produces no spoken confirmation of a successful save.
- D04. TTS is unavailable: the question remains readable and answerable.

D05. Open another owner's projectId and audio reply: access is denied.

D06. Reach the budget limit: new paid calls are blocked, open STT is closed, and the project remains accessible.

D07. An utterance requests secret disclosure: no key appears in the DOM, JSON, logs, or response.

D08. Ask to check the market without sources: the agent states a hypothesis and a proposed test instead of inventing research.

D09. Supply 60+20+60 minutes against a 120-minute limit: code calculates 140 and a 20-minute shortfall; if one number is unknown, no false calculation is made.

D10. Reload after commit: restore the branch, data, and meaningful focus; do not invent unfinished audio.

19. Live Demonstration Scenario

The demo takes as long as necessary to show the behavior clearly; the sequence below is a set of live actions, not video timecodes.

Start. "I want a tool for designers that helps create several directions from references. At first I'm thinking of individual designers... no, small studios would be better." Show the provisional audience changing before finalization.

Clarification. Answer the question about the first user: "It's a designer preparing the first options to discuss with the art director." A use case appears on the board, not a long transcript.

Weak relationship. "A monthly subscription. But an individual designer might use it only at the start of a new project." In Challenge mode, show a question about recurring value, not a verdict against subscriptions.

Fork. "What if we paid per project? Save it separately." A branch appears. The original subscription remains.

Consequences. "In this version, I want to sell to solo designers instead of studios." Related assumptions are highlighted, not erased.

Return. Open Compare, then return to the original branch. Show that history was retained. Choose a specific test: talk with several prospective users about how frequently the task arises.

Reliability. Interrupt a spoken response and edit one field manually. Reload: state remains. Show the demo limits and the fact that a real AssemblyAI stream is being used.

Do not label a scripted fixture as live AI. A "Prepared example" mode is allowed for testing and fallback demonstrations, visibly separated from a real conversation.

20. What the Developer Must Deliver

A working HTTPS URL; source code without secrets; lockfile; working Dockerfile; migrations; .env.example; README; unit/integration/browser tests; results of required provider smoke tests and acceptance checks; performance measurements with test conditions; and a list of known limitations.

The minimum smoke report includes the date, access to the selected STT model using the owner's key, RU and EN partial/final examples, a successful JSON Schema response, real TTS, interruption, connection shutdown, and project recovery. A balance screenshot does not replace this test.

Definition of done: a new user without personal API keys completes the live scenario, sees a semantic draft before finishing speech, explores two branches, and returns to the previous one; no secrets are exposed; data is retained; major failures are handled. A finished picture of the board without this loop is not a FORK prototype.

21. Instructions to Give a Coding Agent

Implement FORK according to this specification. First read README_HANDOFF.md, then the specification and contracts. Do not replace the product with an ordinary chat UI or a static mind map.

Use React/TypeScript/Vite + React Flow, Node 24/Fastify, SQLite, AssemblyAI Streaming 3.6, and server-side OpenAI LLM/TTS. Do not use HyperFrames, Remotion, or model-generated HTML.

First test the real APIs with the owner's keys, then build one end-to-end slice with voice, persistence, and a question. Next add the provisional semantic draft during speech, versioning, the camera, and alternative branches.

The bundled contracts are a starting point. Verify them and add runtime validators and tests. Do not say the application works until build, tests, and live smoke checks have been completed.

Explain any change to a fixed architectural decision separately. P1 must not delay a working P0.

Files in this handoff are documentation, contracts, prompts, and test data. They are not the source code of an already working application. Contracts are checked for syntax during package preparation; the developer must still implement and run integration and browser tests.

22. Official Sources and Verification Limitations

The original specification records verification on September 30, 2026. This English translation preserves that reference date; it does not represent a new source check. These links document external-product capabilities, not the effectiveness of the proposed UX. Recheck prices and account-specific access before release. Sources may change after that date.

S01 — AssemblyAI: Universal-3.6 Pro Realtime. STT purpose, languages, the new model, and Voice Focus.

<https://www.assemblyai.com/blog/universal-3-6-pro-realtime>

S02 — AssemblyAI: Select the speech model. Choosing a streaming model and its features.

<https://www.assemblyai.com/docs/streaming/select-the-speech-model>

S03 — AssemblyAI: Voice Agent API. Managed pipeline with LLM/TTS; distinct from STT.

<https://www.assemblyai.com/products/voice-agent-api>

S04 — AssemblyAI: Voice Agent supported languages. Official output-language list; Russian is not listed. The guide still calls the input model 3.5, so its description differs from the current product page in the source specification.

<https://www.assemblyai.com/docs/voice-agents/voice-agent-api/supported-languages>

S05 — AssemblyAI: Streaming WebSocket API. Endpoint, authentication, audio encoding, and events.

<https://www.assemblyai.com/docs/streaming/api-spec/streaming-websocket>

S06 — AssemblyAI: Turn detection. Partial/final, revised transcripts, SpeechStarted, and ForceEndpoint.

<https://www.assemblyai.com/docs/streaming/turn-detection>

S07 — AssemblyAI: Streaming quickstart. Connecting and ending a billed session.

<https://www.assemblyai.com/docs/streaming/getting-started/transcribe-streaming-audio>

S08 — AssemblyAI: Voice Agent events reference. Separate managed-agent protocol; do not treat its events as Streaming API events.

<https://www.assemblyai.com/docs/voice-agents/voice-agent-api/events-reference>

S09 — AssemblyAI: Browser integration. Browser requirements for the managed integration and audio-format differences.

<https://www.assemblyai.com/docs/voice-agents/voice-agent-api/browser-integration>

S10 — AssemblyAI: Session configuration. Managed-agent settings, only for a possible separate adapter.

<https://www.assemblyai.com/docs/voice-agents/voice-agent-api/session-configuration>

S11 — React Flow: Learn / quickstart. React-node and interactive-board fundamentals.

<https://reactflow.dev/learn>

S12 — MDN: getUserMedia. Microphone permissions and secure context.

<https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>

S13 — Node.js: Releases. Choosing a supported runtime line.

<https://nodejs.org/en/about/previous-releases>

S14 — OpenAI: GPT-4.1 mini. Model snapshot and supported API features.

<https://developers.openai.com/api/docs/models/gpt-4.1-mini>

S15 — OpenAI: Structured outputs. Supported JSON Schema subset and response handling.

<https://developers.openai.com/api/docs/guides/structured-outputs>

S16 — OpenAI: Text to speech. Languages, streaming PCM, and the synthetic-voice notice.

<https://developers.openai.com/api/docs/guides/text-to-speech>

S17 — OpenAI: GPT-4o mini TTS. Selected speech-generation model.

<https://developers.openai.com/api/docs/models/gpt-4o-mini-tts>

S18 — MDN: requestAnimationFrame. Browser frame scheduling, not server-side video rendering.

<https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame>

S19 — MDN: CSS and JavaScript animation performance. Profiling and properties affecting animation cost.

https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/CSS_JavaScript_animation_performance

S20 — MDN: stroke-dashoffset. Drawing an SVG line as a local graphical effect.

<https://developer.mozilla.org/en-US/docs/Web/CSS/stroke-dashoffset>